…and if you're wondering what those backslashes are doing in the pattern: they're escaping another metacharacter: `.` which (without the escape) matches any character except a newline. Let's look at escaping next.

Escape Sequences

We've just met the escape character, backslash, which we used to escape the delimiter metacharacter because we wanted to match a literal forward slash (two of them, in fact). Every other metacharacter (we'll get to those in a moment) that you wish to interpret as a literal character needs to be escaped in the same way. There are other uses for backslash, or in other words, there are different forms of escapes. There are escapes for representing non-printing ASCII characters (newline, tab, control characters) or for using byte values, more or less identically to the way we've used these in strings previously. There are also escape sequences for representing character types such as digits, words (formed from letters), and spaces (with and without directionality). Finally, there are sequences that are used to make simple assertions in the pattern, such as matching a pattern from the very start of the subject text or disabling metacharacters. That's a lot to take in, so let's work up a few examples to demonstrate.

"Non-printing" Characters

Something of a misnomer, "non-printing" characters often have effects on the text in which they're embedded but present a challenge to represent syntactically. You are free to use ASCII byte values with the `\x` sequence (`\x20` is space, `\x0A` is newline, etc.) but the usual convenience escapes (`\n`, `\r`, `\t`, etc.) are available:

```
$text = <<<EOS
hello
world
EOS;
var_dump(preg_match_all('/o\nw/', $text));
```

```
Output:
int(1)
```